



LABORATORIO: FILE PATH TRAVERSAL - SIMPLE CASE

CNO IV - SEGURIDAD INFORMÁTICA

MELANIE ALONDRA ALBA DIEZ
185583
ING. EN TECNOLOGÍAS DE LA INFORMACIÓN

Índice

Especificaciones del laboratorio.....	2
Objetivo del laboratorio	2
Explicación técnica de la vulnerabilidad	2
Procedimiento	3
1. Acceso al laboratorio mediante Burp Suite	3
2. Identificación de las peticiones de imágenes.....	3
3. Envío de la petición a Repeater	4
4. Modificación del parámetro vulnerable	5
5. Análisis de la respuesta	5
6. Confirmación de resolución	6
Explicación del payload.....	7
Mitigación recomendada	7
Referencias	8

Especificaciones del laboratorio

Nombre del laboratorio: File path traversal, simple case

Nivel: APPRENTICE

Tipo de explotación: File Path Traversal (Directory Traversal) mediante manipulación de un parámetro HTTP.

Objetivo del laboratorio

Identificar y explotar de manera controlada una vulnerabilidad de **File Path Traversal** presente en el mecanismo utilizado por una aplicación web para cargar imágenes de productos. Mediante el uso de **Burp Suite**, se busca interceptar y modificar una **petición HTTP** que contiene el parámetro *filename*, manipulando su valor mediante secuencias de recorrido de directorios para acceder a un archivo ubicado fuera del directorio destinado al almacenamiento de imágenes.

El laboratorio se considera resuelto al conseguir que el servidor procese el **payload** `../../../../etc/passwd` y devuelva en la respuesta HTTP el contenido del archivo `/etc/passwd`, demostrando que la aplicación no restringe adecuadamente las rutas proporcionadas por el usuario y permite el acceso no autorizado a archivos del sistema.

Explicación técnica de la vulnerabilidad

File Path Traversal, también conocida como **Directory Traversal**, es una vulnerabilidad que ocurre cuando una aplicación utiliza datos controlables por el usuario para construir una ruta del sistema de archivos sin realizar una validación adecuada. Esto puede permitir que un atacante abandone el directorio al que debería estar limitado y acceda a archivos ubicados en otras partes del servidor. **PortSwigger** señala que este tipo de vulnerabilidad puede exponer archivos de la propia aplicación, credenciales de sistemas backend y archivos sensibles del sistema operativo.

En este laboratorio, las imágenes de los productos se solicitan mediante **peticiones HTTP** al endpoint `/image`, utilizando un parámetro denominado *filename*. Por ejemplo: `GET /image?filename=57.jpg`.

La aplicación utiliza el valor recibido en *filename* para localizar el archivo correspondiente dentro del directorio en el que se almacenan las imágenes. El problema aparece porque dicho parámetro puede ser manipulado por el usuario y la aplicación no comprueba adecuadamente que la ruta resultante permanezca dentro del directorio autorizado.

La secuencia `../` tiene un significado especial en los sistemas de archivos: indica que debe retrocederse un nivel dentro de la estructura de directorios. Por esta razón, al proporcionar varias secuencias consecutivas es posible salir progresivamente del directorio donde se encuentran las imágenes.

En el laboratorio se sustituyó el nombre de la imagen por: `../../../../etc/passwd`.

Al no existir un mecanismo eficaz que impida el recorrido de directorios, el servidor interpreta estas secuencias como parte de una ruta válida y termina accediendo al archivo `/etc/passwd`. En sistemas Unix, este archivo contiene información sobre las cuentas registradas en el sistema.

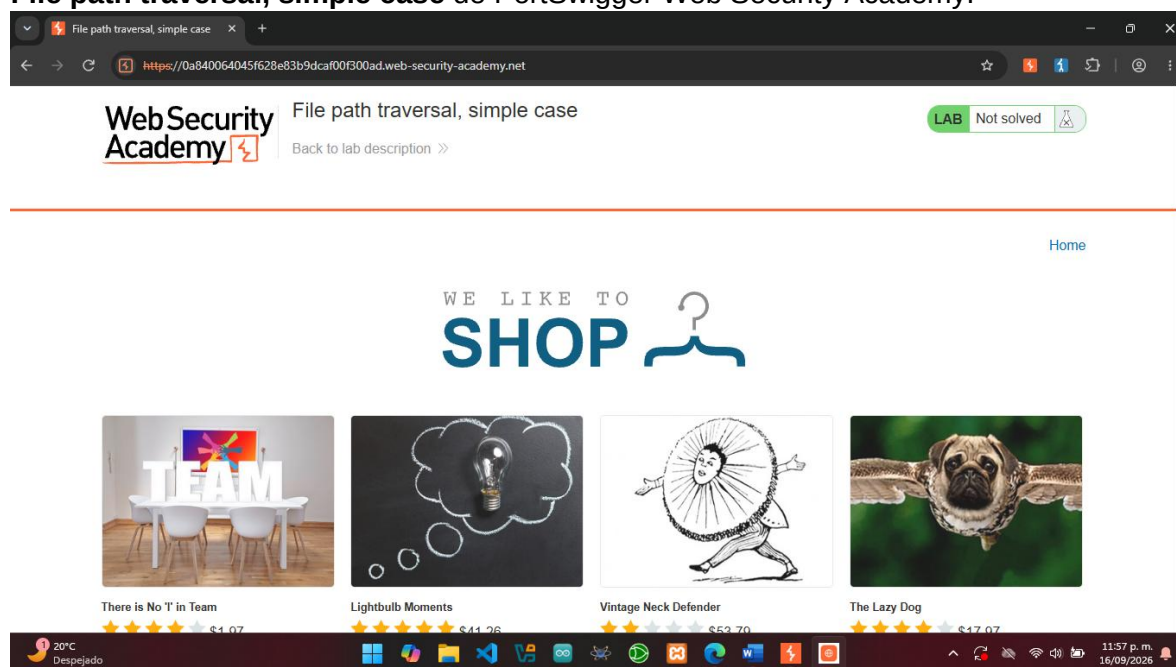
La vulnerabilidad afecta principalmente al principio de **confidencialidad de la información**, debido a que permite consultar archivos que no deberían estar disponibles para el usuario de la aplicación. Dependiendo de los archivos accesibles y de la configuración del servidor, una vulnerabilidad de este tipo puede exponer archivos de configuración, código fuente, credenciales, registros u otra información sensible.

Procedimiento

1. Acceso al laboratorio mediante Burp Suite

Se abrió **Burp Suite Community Edition** y, desde la pestaña **Proxy**, se inició el navegador Chromium integrado mediante la opción **Open Browser**. Este navegador se encuentra configurado para dirigir su tráfico a través del proxy de Burp Suite, permitiendo observar las peticiones y respuestas HTTP generadas durante la navegación.

Posteriormente, desde dicho navegador se accedió a la instancia asignada del laboratorio **File path traversal, simple case** de PortSwigger Web Security Academy.



2. Identificación de las peticiones de imágenes

Desde **Proxy > HTTP history** se analizaron las solicitudes generadas por la aplicación. Se observó que la carga de las imágenes de los productos producía múltiples peticiones **GET** dirigidas al endpoint `/image`, cada una de ellas utilizando el parámetro `filename` para indicar el archivo que debía ser recuperado. Por ejemplo: `GET /image?filename=57.jpg`.

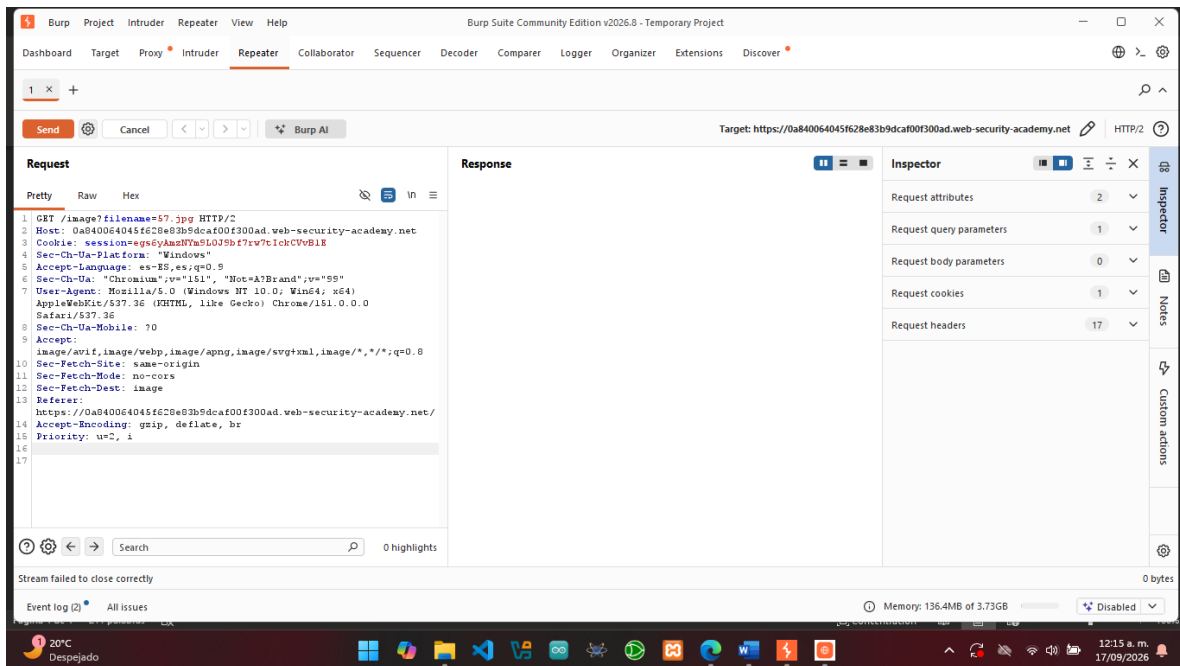
Este comportamiento permitió identificar *filename* como un parámetro susceptible de prueba, debido a que su valor determina el archivo solicitado al servidor.

# ^	Host	Method	URL	Params	Edited	Status code
1	https://0a840064045f628e8...	GET	/			200
4	https://0a840064045f628e8...	GET	/image?filename=57.jpg	✓		200
6	https://0a840064045f628e8...	GET	/image?filename=25.jpg	✓		200
7	https://0a840064045f628e8...	GET	/image?filename=47.jpg	✓		200
8	https://0a840064045f628e8...	GET	/image?filename=37.jpg	✓		200
11	https://0a840064045f628e8...	GET	/image?filename=43.jpg	✓		200
12	https://0a840064045f628e8...	GET	/image?filename=62.jpg	✓		200
14	https://0a840064045f628e8...	GET	/image?filename=8.jpg	✓		200
15	https://0a840064045f628e8...	GET	/image?filename=10.jpg	✓		200
16	https://0a840064045f628e8...	GET	/image?filename=40.jpg	✓		200
17	https://0a840064045f628e8...	GET	/image?filename=66.jpg	✓		200

3. Envío de la petición a Repeater

Se seleccionó una de las peticiones dirigidas a */image* y se utilizó la opción **Send to Repeater**. De esta manera, Burp Suite copió la petición al módulo Repeater, permitiendo modificar manualmente sus parámetros y reenviarla al servidor para analizar las respuestas obtenidas.

The screenshot displays the Burp Suite interface. At the top, the 'Proxy' tab is active, showing the 'HTTP history' tab. A list of intercepted requests is visible, with the request to `/image?filename=57.jpg` selected. A right-click context menu is open over this request, and the 'Send to Repeater' option is highlighted. Below the list, the 'Request' tab is open, showing the raw HTTP request details, including the method (GET), host, and various headers like 'Cookie', 'Sec-Ch-Ua-Platform', 'Accept-Language', 'Sec-Ch-Ua', 'User-Agent', 'AppleWebKit', 'Safari', 'Sec-Ch-Ua-Mobile', 'Accept', 'image/avif, image/vebp, image/apng, image/svg+xml, image/*, */*; q=0.8', 'Sec-Fetch-Site', and 'Sec-Fetch-Mode'.



4. Modificación del parámetro vulnerable

Dentro de **Repeater**, se modificó el valor original del parámetro *filename*, sustituyendo el nombre de la imagen por el siguiente payload: `../../../../etc/passwd`.

Posteriormente se envió la petición modificada al servidor. Las secuencias `../` permiten retroceder niveles dentro de la estructura de directorios, por lo que su utilización buscaba abandonar el directorio destinado al almacenamiento de imágenes y alcanzar el archivo `/etc/passwd`.

5. Análisis de la respuesta

El servidor respondió con: `HTTP/2 200 OK`.

En el cuerpo de la respuesta se observó información correspondiente al archivo `/etc/passwd`. Esto confirmó que el parámetro *filename* podía utilizarse para acceder a archivos situados fuera del directorio previsto por la aplicación.

La recuperación exitosa de `/etc/passwd` confirmó la existencia y explotación de la vulnerabilidad de **File Path Traversal**.

Explicación del payload

El **payload** utilizado para explotar la vulnerabilidad fue: `../../../../etc/passwd`. Este payload puede dividirse en dos componentes principales:

../: Representa una secuencia de recorrido de directorios. Dentro de una ruta, `..` hace referencia al directorio padre y `/` actúa como separador. Por lo tanto, cada aparición de `../` indica al sistema que debe retroceder un nivel dentro de la estructura de directorios.

La repetición de la secuencia tres veces intenta ascender tres niveles desde el directorio base utilizado por la aplicación para almacenar las imágenes.

etc/passwd: Corresponde al archivo objetivo. `/etc/passwd` es un archivo estándar presente en sistemas Unix que almacena información relacionada con las cuentas registradas en el sistema. Su recuperación permite demostrar que la aplicación no está restringiendo la lectura únicamente a los archivos que deberían encontrarse dentro del directorio de imágenes.

Por lo tanto, el payload funciona porque la aplicación confía en el valor recibido mediante *filename* y permite que las secuencias de recorrido sean interpretadas por el sistema de archivos. El ataque no modifica físicamente los directorios ni el archivo; manipula la ruta que la aplicación solicita al sistema operativo.

Mitigación recomendada

La medida más efectiva para prevenir vulnerabilidades de **File Path Traversal** consiste en evitar, siempre que sea posible, que datos proporcionados directamente por el usuario sean utilizados para construir rutas que posteriormente se envían a las API del sistema de archivos. Una alternativa consiste en utilizar identificadores internos para seleccionar recursos previamente definidos, en lugar de permitir que el cliente proporcione directamente el nombre o la ruta del archivo.

Cuando sea necesario aceptar nombres de archivos proporcionados por el usuario, PortSwigger recomienda aplicar varias capas de protección. Primero, debe realizarse una **validación estricta mediante una lista de valores permitidos (allowlist)** o restringir la entrada únicamente a los caracteres que sean necesarios. Después, la aplicación debe construir la ruta utilizando un directorio base controlado por el servidor y **canonicalizar la ruta resultante** mediante las funciones apropiadas del sistema de archivos. Finalmente, debe comprobarse que la ruta canonicalizada continúe comenzando por el directorio base autorizado antes de permitir el acceso al archivo.

No es recomendable depender únicamente de bloquear cadenas como `../`, ya que existen diferentes representaciones y técnicas de evasión que pueden utilizarse frente a filtros incompletos.

Finalmente, debe aplicarse el **principio de mínimo privilegio** al proceso que ejecuta la aplicación web, limitando los archivos y directorios a los que puede acceder. Esta medida no elimina por sí sola la vulnerabilidad, pero reduce el impacto potencial si las demás defensas llegan a fallar.

Referencias

PortSwigger. (s. f.). *Path traversal*. Web Security Academy.

[Path traversal — PortSwigger Web Security Academy](#)

PortSwigger. (s. f.). *File path traversal*. Burp Suite Knowledge Base.

[File path traversal — PortSwigger Knowledge Base](#)

PortSwigger. (s. f.). *Testing for directory traversal vulnerabilities with Burp Suite*. Burp Suite Documentation.

[Testing for directory traversal vulnerabilities — PortSwigger](#)

López Contreras, S. (2026). *CNO IV (T37A): Seguridad Informática. Actividad 07 — Road to Hall of Fame: File Path Traversal, Simple Case*. Universidad Politécnica de San Luis Potosí.